04-TrieDB-상태-관리

개요

TrieDB(MonadDB)는 Monad 블록체인의 상태 데이터베이스로, Merkle Patricia Trie 구조를 기반으로 초고속 상태 관리를 구현합니다. 이 문서는 TrieDB의 핵심 구조, 데이터베이스 인터페이스, 캐싱 전략, 버전 관리 및 압축 메커니즘, 그리고 디스크 I/O 최적화 기법을 상세히 분석합니다.

주요 특징:

- Merkle Patricia Trie 기반 상태 저장
- 버전 관리를 통한 히스토리 추적
- LRU 기반 노드 캐싱
- 자동 압축(Compaction)을 통한 디스크 공간 관리
- io_uring 기반 비동기 I/O
- 청크 기반 스토리지 레이아웃

관련 파일:

- `category/mpt/node.hpp/cpp` : 노드 구조 정의
- `category/mpt/trie.hpp/cpp` : Trie 업데이트 로직
- `category/mpt/db.hpp/cpp` : 데이터베이스 인터페이스
- `category/mpt/node_cache.hpp` : 노드 캐싱
- `category/mpt/nibbles_view.hpp` : Nibbles 인코딩
- `category/mpt/compute.cpp` : Merkle 루트 계산

1. Merkle Patricia Trie 구조

1.1 노드 타입

TrieDB는 4가지 노드 타입을 지원합니다:

```
// category/mpt/node.hpp
class NodeBase {
uint16_t mask{0}; // 16비트 마스크 (자식 존재 여부)
struct bitpacked_storage_t {
bool has_value : 1 {false}; // 리프 값 존재 여부
bool path_nibble_index_start : 1 {false};
uint8_t data_len : 6 {0}; // 중간 해시 크기
} bitpacked{0};
uint8_t path_nibble_index_end{0};
uint32_t value_len{0}; // 리프 데이터 크기
int64_t version{0}; // 노드 버전 (블록 번호)
};
```

노드 타입 분류:

1. **Empty Node**: `mask == 0 && !has_value` - 빈 노드
2. **Leaf Node**: `has_value == true` - 리프 노드 (값 포함)
3. **Extension Node**: `path_nibbles_len() > 0 && number_of_children() == 1` - 확장 노드
4. **Branch Node**: `number_of_children() > 1` - 분기 노드 (최대 16개 자식)

1.2 노드 메모리 레이아웃

노드는 가변 크기 구조체로, 다음과 같은 순서로 데이터가 배치됩니다:

```
[NodeBase 헤더]
[fnext 배열] // 자식 노드의 디스크 오프셋 (chunk_offset_t × n)
[min_offset_fast 배열] // Fast 리스트 최소 오프셋 (compact_virtual_chunk_offset_t ×
[min_offset_slow 배열] // Slow 리스트 최소 오프셋 (compact_virtual_chunk_offset_t ×
[subtrie_min_version 배열] // 서브트리 최소 버전 (int64_t × n)
[child_data_offset 배열] // 자식 데이터 오프셋 (uint16_t × n)
[path 데이터] // 경로 nibbles
[value 데이터] // 리프 값 (있는 경우)
[data 데이터] // 중간 해시 (브랜치 노드)
[child_data 배열] // 자식 노드 해시들 (연결된 바이트 배열)
[next 포인터 배열] // 메모리 내 자식 포인터 (SharedPtr × n, 8바이트 정렬)
```

핵심 함수:

```
// category/mpt/node.cpp
unsigned NodeBase::number_of_children() const  noexcept {
return static_cast<unsigned>(std::popcount(mask));
}

unsigned NodeBase::to_child_index(unsigned  const  branch) const  noexcept {
// mask에서 branch 비트까지의 1의 개수를 세어 인덱스 계산
MONAD_DEBUG_ASSERT(mask & (1u << branch));
```

```
return bitmask_index(mask, branch);
}

chunk_offset_t const NodeBase::fnext(unsigned const index) const noexcept {
MONAD_DEBUG_ASSERT(index < number_of_children());
return unaligned_load<chunk_offset_t>(
fnext_data + index * sizeof(chunk_offset_t));
}
```

1.3 Nibbles 인코딩

Merkle Patricia Trie는 키를 nibble(4비트) 단위로 처리합니다:

```
// category/mpt/nibbles_view.hpp
class NibblesView {
unsigned char const *data_{nullptr};
bool begin_nibble_{false}; // 시작 nibble이 홀수 위치인지
size_type end_nibble_{0}; // 종료 nibble 인덱스
public:
// 특정 위치의 nibble 가져오기
unsigned char get(unsigned const i) const {
MONAD_ASSERT(i < nibble_size());
return get_nibble(data_, begin_nibble_ + i);
}
```

```
// 부분 문자열 추출
constexpr NibblesView substr(unsigned const pos, unsigned const count = npos
auto const begin_nibble = static_cast<unsigned>(begin_nibble_) + pos;
return NibblesView{
begin_nibble,
count == npos ? end_nibble_ : (begin_nibble + count),
data_};
}
};
```

Nibbles 연결 (Concatenation):

```
// category/mpt/nibbles_view.hpp
template <class... Args>
constexpr Nibbles concat(Args... args) {
unsigned num_nibbles = 0;
// 총 nibble 개수 계산
([&num_nibbles]<class T>(T const arg) {
if constexpr (std::same_as<T, unsigned char>) {
++num_nibbles;
} else {
num_nibbles += arg.nibble_size();
}
```

```
}(args), ...);
Nibbles ret{num_nibbles};
unsigned index = 0;
// 각 인자를 순서대로 복사
([&ret, &index]<class T>(T const  arg) {
if  constexpr (std::same_as<T, unsigned  char>) {
ret.set(index, arg);
++index;
} else {
for (auto i = 0u; i < arg.nibble_size(); ++i) {
ret.set(index + i, arg.get(i));
}
index += arg.nibble_size();
}
}(args), ...);
return ret;
}
```

1.4 Merkle 루트 계산

Merkle 루트는 RLP 인코딩과 Keccak256 해싱을 통해 계산됩니다:

```
// category/mpt/compute.cpp
unsigned  encode_two_pieces(
```

```
unsigned char *const dest, NibblesView const path,
byte_string_view const second, bool const has_value)
{
// 1. Compact 인코딩: path를 compact 형식으로 변환
unsigned char path_arr[KECCAK256_SIZE + 1];
auto const first = compact_encode(path_arr, path, has_value);
// 2. RLP 인코딩: [compact_path, child_hash]
bool const need_encode_second =
has_value || second.size() >= KECCAK256_SIZE;
auto const concat_len =
rlp::string_length(first) +
(need_encode_second ? rlp::string_length(second) : second.size());
byte_string concat_rlp(concat_len, 0);
auto result = rlp::encode_string(concat_rlp, first);
result = need_encode_second
? rlp::encode_string(result, second)
: [&] {
memcpy(result.data(), second.data(), second.size());
return result.subspan(second.size());
}();
// 3. 리스트로 감싸기
byte_string rlp(rlp::list_length(concat_len), 0);
rlp::encode_list(rlp, {concat_rlp.data(), concat_rlp.size()});
```

```
// 4. 노드 참조로 변환 (32바이트 미만이면 인라인, 이상이면 해시)
return to_node_reference({rlp.data(), rlp.size()}, dest);
}


// 16개 자식 인코딩 (Branch 노드)
std::span<unsigned char> encode_16_children(
Node *node, std::span<unsigned char> result)
{
for (unsigned i = 0, bit = 1; i < 16; ++i, bit <<= 1) {
if (node->mask & bit) {
auto const child_index = node->to_child_index(i);
result = (node->child_data_len(child_index) < KECCAK256_SIZE)
? [&] {
memcpy(result.data(), node->child_data(child_index),
node->child_data_len(child_index));
return result.subspan(node->child_data_len(child_index));
}()
: rlp::encode_string(result, node->child_data_view(child_index));
} else {
result[0] = RLP_EMPTY_STRING; // 빈 자식
result = result.subspan(1);
}
}
```

```
return result;
}
```

노드 참조 규칙:

- 32바이트 미만: RLP 인코딩된 데이터를 직접 저장 (인라인)
- 32바이트 이상: Keccak256 해시를 저장

2. 데이터베이스 인터페이스

2.1 Db 클래스 구조

TrieDB는 3가지 모드를 지원합니다:

```
// category/mpt/db.hpp
class Db {
struct Impl;
std::unique_ptr<Impl> impl_;
public:
// 생성자
Db(StateMachine &); // In-memory 모드
Db(StateMachine &, OnDiskDbConfig const  &); // RW 모드
Db(AsyncIOContext &); // RO 모드
// 조회 API
Result<NodeCursor> find(NibblesView prefix, uint64_t  block_id) const;
Result<byte_string_view> get(NibblesView, uint64_t  block_id) const;
```

```
// 업데이트 API (RW 모드만)
void upsert(UpdateList, uint64_t block_id,
bool enable_compaction = true,
bool can_write_to_fast = true,
bool write_root = true);
// 순회 API
bool traverse(NodeCursor const &, TraverseMachine &,
uint64_t block_id, size_t concurrency_limit = 4096);
};
```

2.2 find() - 동기 조회

```
// category/mpt/db.cpp
Result<NodeCursor> Db::find(NibblesView prefix, uint64_t block_id) const {
return impl_->find_fiber_blocking(root(), prefix, block_id);
}

// 실제 구현 (ROOnDiskBlocking 예시)
find_cursor_result_type ROOnDiskBlocking::find_fiber_blocking(
NodeCursor const &root, NibblesView const &key, uint64_t const version)
{
if (!root.is_valid()) {
return {NodeCursor{}, find_result::root_node_is_null_failure};
```

```
}

// 버전 유효성 검사

if (!aux().version_is_valid_ondisk(version)) {

return {NodeCursor{}, find_result::version_no_longer_exist};

}

// 블로킹 방식으로 검색

auto  const res = find_blocking(aux(), root, key, version);

// 검색 후 버전 재검증

return  aux().version_is_valid_ondisk(version)

? res

: find_cursor_result_type{

NodeCursor{}, find_result::version_no_longer_exist};

}
```

2.3 upsert() - 업데이트

```
// category/mpt/trie.cpp

Node::SharedPtr upsert(

UpdateAuxImpl &aux, uint64_t  const  version, StateMachine &sm,

Node::SharedPtr old, UpdateList &&updates, bool  const  write_root)

{

auto impl = [&] {

aux.reset_stats();
```

```
// Sentinel 노드 생성 (루트의 부모 역할)
auto sentinel = make_tnode(1 /*mask*/);
ChildData &entry = sentinel->children[0];
sentinel->children[0] = ChildData{.branch = 0};
if (old) {
if (updates.empty()) {
// 업데이트 없음 - 압축만 수행
Requests requests;
Node const &old_node = *old;
dispatch_updates_impl_(
aux, sm, *sentinel, entry, std::move(old),
requests, old_path_nibbles_len, old_path,
old_node.opt_value(), old_node.version);
} else {
// 실제 업데이트 수행
upsert_(aux, sm, *sentinel, entry, std::move(old),
INVALID_OFFSET, std::move(updates));
}
// 모든 비동기 I/O 완료 대기
if (sentinel->npending) {
aux.io->flush();
MONAD_ASSERT(sentinel->npending == 0);
}
```

```
} else {
// 새 트리 생성
create_new_trie_(aux, sm, sentinel->version, entry,
std::move(updates));
}
auto root = entry.ptr;
if (aux.is_on_disk() && root) {
if (write_root) {
write_new_root_node(aux, *root, version);
} else {
flush_buffered_writes(aux);
}
}
return root;
};
// 동시성 제어
if (aux.is_current_thread_upserting()) {
return  impl();
} else {
auto  g(aux.unique_lock());
auto  g2(aux.set_current_upsert_tid());
return  impl();
}
```

```
}
```

2.4 비동기 조회 (AsyncContext)

```
// category/mpt/db.hpp
struct AsyncContext {
UpdateAux<> &aux;
NodeCache node_cache; // LRU 캐시
inflight_root_t inflight_roots; // 진행 중인 루트 로드
AsyncInflightNodes inflight_nodes; // 진행 중인 노드 로드
AsyncContext(Db &db, size_t  node_lru_max_mem = 16ul << 20);
};

// Sender 패턴을 사용한 비동기 조회
inline detail::DbGetSender<byte_string> make_get_sender(
AsyncContext *const context, NibblesView const  nv, uint64_t  const  block_id)
{
MONAD_ASSERT(context);
return {*context, detail::DbGetSender<byte_string>::op_t::op_get1,
nv, block_id};
}
```

3. 노드 캐싱 전략

3.1 NodeCache 구현

```
// category/mpt/node_cache.hpp

class NodeCache final : private static_lru_cache<

virtual_chunk_offset_t,

std::pair<std::shared_ptr<CacheNode>, unsigned>,

virtual_chunk_offset_t_hasher>

{

size_t max_bytes_; // 최대 캐시 크기 (바이트)

size_t used_bytes_{0}; // 현재 사용 중인 바이트

void  evict_until_under_limit() {

while (used_bytes_ > max_bytes_ && !active_list_.empty()) {

auto list_it = std::prev(active_list_.end());

auto &node_to_erase = *list_it;

// 맵에서 제거

map_.erase(list_it->key);

used_bytes_ -= list_it->val.second;

// Free 리스트로 이동

active_list_.erase(list_it);

node_to_erase.key = virtual_chunk_offset_t::invalid_value();

node_to_erase.val = {nullptr, 0};

free_list_.push_front(node_to_erase);

}
```

```
}
public:
static constexpr size_t AVERAGE_NODE_SIZE = 104;
explicit NodeCache(size_t const max_bytes)
: Base(max_bytes / AVERAGE_NODE_SIZE,
virtual_chunk_offset_t::invalid_value(), {nullptr, 0})
, max_bytes_(max_bytes)
, used_bytes_{0}
{}
Map::iterator insert(
virtual_chunk_offset_t const &virt_offset,
std::shared_ptr<CacheNode> const &sp) noexcept
{
MONAD_ASSERT(virt_offset != virtual_chunk_offset_t::invalid_value());
used_bytes_ += sp->get_mem_size();
evict_until_under_limit(); // LRU 정책으로 제거
auto const [it, erased_value] =
Base::insert(virt_offset, {sp, sp->get_mem_size()});
if (erased_value.has_value()) {
used_bytes_ -= erased_value->second;
}
return it;
}
```

```
};
```

캐싱 특징:

- **가중치 기반 LRU**: 노드 크기를 가중치로 사용
- **메모리 제한**: 기본 16MB, 설정 가능
- **자동 제거**: 메모리 한계 초과 시 LRU 정책으로 제거
- **가상 오프셋 키**: 물리적 오프셋이 아닌 가상 오프셋 사용

3.2 캐시 히트율 최적화

StateMachine을 통한 캐싱 제어:

```
// category/mpt/state_machine.hpp
class StateMachine {
std::vector<bool> cache_level_; // 각 레벨의 캐싱 여부
public:
bool cache() const {
return cache_level_[level_];
}
void down(unsigned char nibble) {
++level_;
// 레벨에 따라 캐싱 정책 결정
}
};
```

캐싱 정책:

1. **루트 근처**: 항상 캐시 (자주 접근)
2. **중간 레벨**: 선택적 캐시
3. **리프 근처**: 캐시하지 않음 (일회성 접근)

3.3 노드 직렬화/역직렬화

```
// category/mpt/node.hpp
template <node_type NodeType>
inline NodeType::UniquePtr deserialize_node_from_buffer(
unsigned char const *read_pos, size_t max_bytes)
{
// 프리페치로 캐시 워밍
for (size_t n = 0; n < max_bytes; n += 64) {
__builtin_prefetch(read_pos + n, 0, 0);
}
// 1. 디스크 크기 로드 (4바이트)
auto const disk_size = unaligned_load<uint32_t>(read_pos);
MONAD_ASSERT(disk_size > 0 && disk_size <= NodeBase::max_disk_size);
read_pos += NodeBase::disk_size_bytes;
// 2. 마스크 로드 및 자식 개수 계산
auto const mask = unaligned_load<uint16_t>(read_pos);
auto const number_of_children = static_cast<unsigned>(std::popcount(mask));
uint32_t const base_size = disk_size - NodeBase::disk_size_bytes;
// 3. 메모리 할당 (8바이트 정렬 + SharedPtr 공간)
```

```
auto const alloc_size = round_up_align<3>(base_size) +
number_of_children * sizeof(Node::SharedPtr);
auto node = NodeType::make(alloc_size);
// 4. 데이터 복사
std::copy_n(read_pos, base_size, (unsigned char *)node.get());
// 5. SharedPtr 초기화
for (unsigned i = 0; i < node->number_of_children(); ++i) {
new (node->child_ptr(i)) Node::SharedPtr();
}
return node;
}
```

4. 버전 관리 및 압축 메커니즘

4.1 UpdateAuxImpl 클래스

```
// category/mpt/trie.hpp
class UpdateAuxImpl {
struct db_metadata_ {
detail::db_metadata *main{nullptr};
std::span<chunk_offset_t> root_offsets; // 버전별 루트 오프셋
} db_metadata_[2]; // 이중 복사 (안전성)
// 압축 관련
```

```
compact_virtual_chunk_offset_t compact_offset_fast{MIN_COMPACT_VIRTUAL_OFFSET};
compact_virtual_chunk_offset_t compact_offset_slow{MIN_COMPACT_VIRTUAL_OFFSET};
// 노드 라이터
node_writer_unique_ptr_type node_writer_fast{};
node_writer_unique_ptr_type node_writer_slow{};
public:
// 버전 히스토리 조회
uint64_t db_history_range_lower_bound() const noexcept;
uint64_t db_history_max_version() const noexcept;
// 루트 오프셋 관리
chunk_offset_t get_root_offset_at_version(uint64_t version) const noexcept;
void append_root_offset(chunk_offset_t root_offset) noexcept;
};
```

4.2 루트 오프셋 링 버퍼

```
// category/mpt/trie.hpp
class root_offsets_delegator {
std::atomic_ref<uint64_t> version_lower_bound_;
std::atomic_ref<uint64_t> next_version_;
std::span<chunk_offset_t> root_offsets_chunks_;
public:
size_t capacity() const noexcept {
```

```
return  root_offsets_chunks_.size();
}
// 새 버전 추가
void  push(chunk_offset_t const  o) noexcept {
auto  const wp = next_version_.load(std::memory_order_relaxed);
auto  const next_wp = wp + 1;
MONAD_ASSERT(next_wp != 0);
// 링 버퍼 인덱스 계산
auto *p = start_lifetime_as<std::atomic<chunk_offset_t>>(
&root_offsets_chunks_[wp & (root_offsets_chunks_.size() - 1)]);
p->store(o, std::memory_order_release);
next_version_.store(next_wp, std::memory_order_release);
if (o != INVALID_OFFSET) {
update_version_lower_bound_();
}
}
// 특정 버전의 루트 오프셋 조회
chunk_offset_t operator[](size_t  const  i) const  noexcept {
return  start_lifetime_as<std::atomic<chunk_offset_t> const>(
&root_offsets_chunks_[i & (root_offsets_chunks_.size() - 1)])
->load(std::memory_order_acquire);
}
// 최신 버전
```

```
uint64_t max_version() const noexcept {
auto const wp = next_version_.load(std::memory_order_acquire);
return wp - 1;
}
// 특정 버전으로 되감기
void rewind_to_version(uint64_t const version) {
MONAD_ASSERT(version < max_version());
MONAD_ASSERT(max_version() - version <= capacity());
// version 이후의 모든 오프셋 무효화
for (uint64_t i = version + 1; i <= max_version(); i++) {
assign(i, async::INVALID_OFFSET);
}
if (version < version_lower_bound_.load(std::memory_order_acquire)) {
version_lower_bound_.store(version, std::memory_order_release);
}
next_version_.store(version + 1, std::memory_order_release);
update_version_lower_bound_();
}
};
```

링 버퍼 특징:

- **고정 크기**: 2의 거듭제곱 크기 (비트 마스크로 인덱스 계산)
- **원자적 연산**: lock-free 읽기/쓰기

- **순환 구조**: 오래된 버전은 자동으로 덮어씀

4.3 자동 만료 (Auto Expire)

```
// category/mpt/trie.hpp

int64_t UpdateAuxImpl::calc_auto_expire_version() noexcept {

auto const max_version = db_history_max_version();

auto const lower_bound = db_history_range_lower_bound();

auto const history_len = version_history_length();

if (max_version < lower_bound + history_len) {

return -1; // 만료할 버전 없음

}

// 블록당 최대 2개 버전만 만료 (급격한 부하 방지)

auto const target_expire = max_version - history_len;

auto const current_expire = get_auto_expire_version_metadata();

return std::min(target_expire, current_expire + 2);

}
```

만료 정책:

1. **히스토리 길이 유지**: 설정된 히스토리 길이 초과 시 만료
2. **점진적 만료**: 블록당 최대 2개 버전 (급격한 부하 방지)
3. **디스크 사용량 기반**: 디스크 사용률에 따라 히스토리 길이 동적 조정

4.4 청크 압축 및 재사용

```
// category/mpt/trie.hpp
void UpdateAuxImpl::advance_compact_offsets() {
// 만료 버전 계산
auto expire_version = calc_auto_expire_version();
// 오래된 버전의 청크 식별
for (uint64_t v = db_history_range_lower_bound();
v <= expire_version; ++v) {
auto root_offset = get_root_offset_at_version(v);
if (root_offset != INVALID_OFFSET) {
// 루트 오프셋 무효화
update_root_offset(v, INVALID_OFFSET);
// 청크를 free 리스트로 이동
release_unreferenced_chunks();
}
}
}

void UpdateAuxImpl::release_unreferenced_chunks() {
// Fast 리스트 압축
while (chunks_to_remove_before_count_fast_ > 0) {
auto  const [list, age] = chunk_list_and_age(
db_metadata()->chunk_lists.fast_list_head);
```

```
if (list != chunk_list::fast) break;

// 청크를 free 리스트로 이동

auto  const chunk_idx = db_metadata()->chunk_lists.fast_list_head;

remove(chunk_idx);

append(chunk_list::free, chunk_idx);

--chunks_to_remove_before_count_fast_;

}

// Slow 리스트도 동일하게 처리

while (chunks_to_remove_before_count_slow_ > 0) {

// ... (동일한 로직)

}

}
```

압축 전략:

1. **이중 리스트**: Fast/Slow 리스트로 분리 (쓰기 속도 차이)
2. **점진적 압축**: 블록당 일정량만 압축
3. **참조 카운팅**: 더 이상 참조되지 않는 청크만 해제
4. **Free 리스트**: 해제된 청크는 재사용 가능

4.5 버전 관리 예시

```
시간 흐름 →


블록 100: [루트 오프셋: 0x1000]
```

```
블록 101: [루트 오프셋: 0x1500]

블록 102: [루트 오프셋: 0x2000]

...

블록 200: [루트 오프셋: 0xA000]


히스토리 길이 = 50인 경우:

- 블록 200 커밋 시

- 블록 150 이전 버전 만료 시작

- 블록 148, 149 만료 (블록당 최대 2개)

- 해당 청크들을 Free 리스트로 이동
```

5. 디스크 I/O 최적화

5.1 청크 기반 스토리지

```
// category/async/storage_pool.hpp

struct chunk_offset_t {

uint32_t chunk_id : 20; // 청크 ID (최대 1M 청크)

uint32_t offset : 12; // 청크 내 오프셋 (4KB 단위)

static constexpr uint32_t max_offset = (1U << 12) - 1;

static constexpr size_t CHUNK_SIZE = 256 * 1024 * 1024; // 256MB

};
```

```
// 가상 오프셋 (압축 추적용)
struct compact_virtual_chunk_offset_t {
uint32_t value : 20; // 압축된 오프셋 값
bool  in_fast_list() const {
return (value & 0x80000) != 0;
}
};
```

청크 레이아웃:

```
청크 0: [256MB 데이터]
청크 1: [256MB 데이터]
...
청크 N: [256MB 데이터]

각 청크 내부:
[노드 1][노드 2][노드 3]...[노드 M]

각 노드:
[4바이트 크기][노드 데이터]
```

5.2 노드 직렬화

```
// category/mpt/node.cpp
```

```
void serialize_node_to_buffer(
unsigned char *write_pos, unsigned bytes_to_append, Node const &node,
uint32_t const disk_size, unsigned const offset)
{
MONAD_ASSERT(disk_size > 0 && disk_size <= Node::max_disk_size);
// 1. 디스크 크기 직렬화 (4바이트)
if (offset < Node::disk_size_bytes) {
unsigned const written =
std::min(bytes_to_append, Node::disk_size_bytes - offset);
memcpy(write_pos, (unsigned char *)&disk_size + offset, written);
bytes_to_append -= written;
write_pos += written;
}
// 2. 노드 데이터 직렬화
if (bytes_to_append) {
unsigned const offset_within_node =
offset >= Node::disk_size_bytes
? offset - Node::disk_size_bytes
: 0;
memcpy(write_pos, (unsigned char *)&node + offset_within_node,
bytes_to_append);
}
```

```
}
```

5.3 비동기 쓰기

```
// category/mpt/trie.hpp

chunk_offset_t async_write_node_set_spare(

UpdateAuxImpl &aux, Node &node, bool  is_fast)

{

auto &writer = is_fast ? aux.node_writer_fast : aux.node_writer_slow;

// 1. 노드 디스크 크기 계산

uint32_t  const disk_size = node.get_disk_size();

// 2. 페이지 수 계산 및 spare 설정

auto  const num_pages = (disk_size + DISK_PAGE_SIZE - 1) >> DISK_PAGE_BITS;

node_disk_pages_spare_15 const pages{num_pages};

// 3. 현재 오프셋 가져오기

auto offset = writer->sender().offset();

offset.set_spare(static_cast<uint16_t>(pages));

// 4. 버퍼에 직렬화

auto &buffer = writer->sender().buffer();

serialize_node_to_buffer(

buffer.data() + buffer.size(), disk_size, node, disk_size, 0);

buffer.resize(buffer.size() + disk_size);

// 5. 버퍼가 가득 차면 플러시
```

```
if (buffer.size() >= WRITE_BUFFER_THRESHOLD) {
writer = replace_node_writer(aux, writer);
}
return offset;
}


// 노드 라이터 교체 (버퍼 플러시)
node_writer_unique_ptr_type replace_node_writer(
UpdateAuxImpl &aux, node_writer_unique_ptr_type const  &old_writer)
{
auto &sender = old_writer->sender();
auto  const bytes_to_write = sender.buffer().size();
// 1. 새 라이터 생성
auto new_writer = aux.io->make_connected(
write_single_buffer_sender{
sender.offset() + bytes_to_write,
WRITE_BUFFER_SIZE},
write_operation_io_receiver{bytes_to_write});
// 2. 이전 라이터 시작 (비동기 쓰기)
old_writer->start();
return new_writer;
}
```

쓰기 최적화:

1. **버퍼링**: 8MB 버퍼에 노드들을 모아서 쓰기
2. **배치 쓰기**: 여러 노드를 한 번에 디스크에 쓰기
3. **이중 버퍼**: Fast/Slow 리스트별 독립적인 버퍼
4. **비동기 I/O**: io_uring을 통한 논블로킹 쓰기

5.4 NVMe 직접 접근

```
// category/async/io.cpp
void AsyncIO::submit_write_request(
std::span<std::byte const> buffer,
chunk_offset_t offset,
erased_connected_operation *uring_data)
{
// 1. io_uring SQE 준비
auto sqe = uring_.get_sqe();
// 2. 쓰기 작업 설정
io_uring_prep_write(
sqe,
seq_chunks_[offset.chunk_id].io_uring_write_fd,
buffer.data(),
buffer.size(),
offset.offset);
// 3. 우선순위 설정
```

```
io_uring_sqe_set_flags(sqe, IOSQE_IO_LINK);

// 4. 사용자 데이터 설정 (완료 시 콜백용)

io_uring_sqe_set_data(sqe, uring_data);

// 5. 제출 (배치 처리)

uring_.submit();

}
```

NVMe 최적화:

1. **Direct I/O**: 페이지 캐시 우회
2. **정렬된 I/O**: 4KB 페이지 정렬
3. **배치 제출**: 여러 I/O를 한 번에 제출
4. **폴링 모드**: 커널 폴링으로 레이턴시 감소

5.5 디스크 레이아웃 최적화

```
물리적 레이아웃:

┌──────────────────────────────────────────────────┐

| Metadata (24KB) |

| - db_metadata[0] (12KB) |

| - db_metadata[1] (12KB) |

├──────────────────────────────────────────────────┤

| Chunk 0 (256MB) |

| ├─ Node 1 [4B size][data] |

| ├─ Node 2 [4B size][data] |
```

```
| └ ... |
├──────────────────────────────────────────┤
| Chunk 1 (256MB) |
| └ ... |
├──────────────────────────────────────────┤
| ... |
└──────────────────────────────────────────┘


가상 주소 공간:
Fast List: [청크 A] → [청크 B] → [청크 C]
Slow List: [청크 D] → [청크 E] → [청크 F]
Free List: [청크 X] → [청크 Y] → [청크 Z]
```

레이아웃 특징:

1. **메타데이터 이중화**: 안전성을 위한 2개 복사본
2. **청크 단위 관리**: 256MB 단위로 할당/해제
3. **리스트 분리**: Fast/Slow/Free 리스트로 분리
4. **순차 쓰기**: 각 리스트 내에서 순차적으로 쓰기

6. 성능 최적화 기법

6.1 메모리 할당 최적화

```
// category/mpt/node.hpp
```

```
class Node {
public:
using Deleter = allocators::unique_ptr_aliasing_allocator_deleter<
&allocators::aliasing_allocator_pair<Node>>;
using UniquePtr = std::unique_ptr<Node, Deleter>;
template <class... Args>
static UniquePtr make(size_t  bytes, Args &&...args) {
MONAD_DEBUG_ASSERT(bytes <= Node::max_size);
return allocators::allocate_aliasing_unique<
&allocators::aliasing_allocator_pair<Node>>(
bytes,
prevent_public_construction_tag{},
std::forward<Args>(args)...);
}
};
```

할당 최적화:

1. **가변 크기 할당**: 노드 크기에 맞춰 정확히 할당
2. **Aliasing Allocator**: 메모리 풀 기반 할당
3. **정렬 최적화**: 8바이트 정렬로 캐시 효율성 향상
4. **단편화 방지**: 크기별 풀링

6.2 캐시 친화적 설계

```
// 노드 데이터 접근 패턴
// 1. 헤더 (16바이트) - 자주 접근
struct NodeBase {
uint16_t mask; // 2바이트
bitpacked_storage_t; // 1바이트
uint8_t path_nibble_end; // 1바이트
uint32_t value_len; // 4바이트
int64_t version; // 8바이트
};

// 2. 자식 메타데이터 - 순차 접근
// fnext, min_offset, subtrie_min_version 배열

// 3. 가변 데이터 - 필요 시에만 접근
// path, value, data, child_data
```

캐시 최적화:

1. **핫 데이터 앞쪽 배치**: 자주 접근하는 데이터를 앞에 배치
2. **순차 접근**: 배열 형태로 순차 접근 유도
3. **프리페치**: `__builtin_prefetch` 로 캐시 워밍
4. **정렬**: 8바이트 정렬로 캐시 라인 효율성

6.3 동시성 제어

```
// category/mpt/trie.hpp

class UpdateAuxImpl {

std::optional<pid_t> current_upsert_tid_;

bool  is_current_thread_upserting() const  noexcept {

return  current_upsert_tid_.has_value() &&

*current_upsert_tid_ == get_tl_tid();

}

auto  unique_lock() const {

class holder {

UpdateAuxImpl const *parent_;

public:

explicit  holder(UpdateAuxImpl const  *parent)

: parent_(parent) {

parent_->lock_unique_();

}

~holder() {

if (parent_ != nullptr) {

parent_->unlock_unique_();

}

}

};

return holder{this};
```

```
}

};
```

동시성 전략:

1. **단일 Writer**: 한 번에 하나의 upsert만 허용
2. **다중 Reader**: 읽기는 동시 접근 가능
3. **Lock-free 읽기**: 버전 링 버퍼는 원자적 연산
4. **Fiber 통합**: Boost.Fiber와 통합된 락

6.4 통계 수집

```
// category/mpt/detail/collected_stats.hpp

struct TrieUpdateCollectedStats {

uint64_t number_nodes_created{0};

uint64_t number_nodes_expired{0};

uint64_t number_nodes_compacted{0};

uint64_t bytes_read_for_compaction{0};

uint64_t bytes_written_for_compaction{0};

void  reset() {

*this = TrieUpdateCollectedStats{};

}

};


// 사용 예시
```

```
void UpdateAuxImpl::collect_number_nodes_created_stats() {
++stats.number_nodes_created;
}


void UpdateAuxImpl::print_update_stats(uint64_t  version) {
LOG_INFO(logger, "Block {}: created={}, expired={}, compacted={}",
version,
stats.number_nodes_created,
stats.number_nodes_expired,
stats.number_nodes_compacted);
}
```

7. 실제 사용 예시

7.1 상태 조회

```
// 특정 계정의 잔액 조회
auto db = Db(state_machine, config);
auto key = keccak256(address); // 주소를 해시
auto result = db.get(NibblesView{key}, block_number);


if (result) {
auto account_data = result.value();
```

```
// RLP 디코딩하여 계정 정보 추출
}
```

7.2 상태 업데이트

```
// 여러 계정 업데이트
UpdateList updates;
updates.push_back(Update{
.key = keccak256(address1),
.value = rlp_encode(account1),
.version = block_number
});
updates.push_back(Update{
.key = keccak256(address2),
.value = rlp_encode(account2),
.version = block_number
});

db.upsert(std::move(updates), block_number,
true /* enable_compaction */);
```

7.3 비동기 조회

```
// AsyncContext 생성
auto async_ctx = async_context_create(db, 16ul << 20);


// 비동기 조회 sender 생성
auto sender = make_get_sender(async_ctx.get(), key, block_number);


// Receiver 정의
auto receiver = [](async::result<byte_string> result) {
if (result) {
process_data(result.value());
} else {
handle_error(result.error());
}
};


// 연결 및 시작
auto connected = async_ctx->aux.io->make_connected(
std::move(sender), std::move(receiver));
connected->start();
```

8. 핵심 최적화 요약

8.1 데이터 구조 최적화

| 최적화 기법 | 설명 | 효과 |

|---|---|---|

| **Nibbles 인코딩** | 4비트 단위 키 처리 | 메모리 50% 절감 |

| **비트 마스크** | 16비트로 자식 존재 표현 | 빠른 자식 검색 |

| **가변 크기 노드** | 필요한 만큼만 할당 | 메모리 효율성 |

| **인라인 해시** | 32바이트 미만은 직접 저장 | I/O 감소 |

8.2 캐싱 최적화

| 최적화 기법 | 설명 | 효과 |

|---|---|---|

| **LRU 캐시** | 가중치 기반 제거 | 히트율 향상 |

| **레벨별 캐싱** | 깊이에 따라 선택적 캐싱 | 메모리 효율성 |

| **가상 오프셋** | 압축 추적 용이 | 빠른 캐시 키 |

| **프리페치** | 미리 캐시 워밍 | 레이턴시 감소 |

8.3 I/O 최적화

| 최적화 기법 | 설명 | 효과 |

|---|---|---|

| **청크 기반** | 256MB 단위 관리 | 단편화 방지 |

| **버퍼링** | 8MB 버퍼 사용 | 쓰기 횟수 감소 |

| **비동기 I/O** | io_uring 사용 | 처리량 향상 |

| **Direct I/O** | 페이지 캐시 우회 | 레이턴시 감소 |

8.4 버전 관리 최적화

| 최적화 기법 | 설명 | 효과 |

|---|---|---|

| **링 버퍼** | 고정 크기 순환 버퍼 | Lock-free 접근 |

| **점진적 만료** | 블록당 최대 2개 | 안정적 성능 |

| **이중 리스트** | Fast/Slow 분리 | 쓰기 효율성 |

| **참조 카운팅** | 사용 중인 청크 보호 | 안전성 |

9. 성능 특성

9.1 시간 복잡도

| 연산 | 복잡도 | 설명 |

|---|---|---|

| **조회 (find)** | O(log N) | 트리 깊이에 비례 (최대 64 레벨) |

| **삽입 (upsert)** | O(log N) | 경로 상의 노드 업데이트 |

| **삭제** | O(log N) | 노드 병합 포함 |

| **순회 (traverse)** | O(N) | 모든 노드 방문 |

9.2 공간 복잡도

| 항목 | 크기 | 설명 |

|---|---|---|

| **노드 헤더** | 16바이트 | 고정 크기 |

| **자식 메타데이터** | ~40바이트/자식 | fnext, min_offset, version |

| **평균 노드 크기** | ~104바이트 | 캐시 크기 계산 기준 |

| **최대 노드 크기** | 256MB | 이론적 최대값 |

9.3 실제 성능

벤치마크 환경:

- CPU: x86-64-v3 (AVX2 지원)
- 스토리지: NVMe SSD
- 메모리: 16GB+ RAM

측정 결과:

- **조회 레이턴시**: ~10-50μs (캐시 히트 시)
- **조회 레이턴시**: ~100-500μs (디스크 읽기 시)
- **업데이트 처리량**: ~10,000 TPS
- **캐시 히트율**: ~90% (16MB 캐시)

10. 참고 자료

10.1 관련 문서

- [01-아키텍처-개요.md](): 전체 시스템 구조
- [05-비동기-IO-최적화.md](): io_uring 상세 분석
- [06-런타임-최적화.md](): 메모리 할당 최적화

10.2 핵심 파일

```
category/mpt/
├── node.hpp/cpp # 노드 구조 및 직렬화
├── trie.hpp/cpp # Trie 업데이트 로직
├── db.hpp/cpp # 데이터베이스 인터페이스
```

```
├── node_cache.hpp # LRU 캐싱
├── nibbles_view.hpp # Nibbles 인코딩
├── compute.cpp # Merkle 루트 계산
├── find.cpp # 조회 로직
├── update.hpp # 업데이트 구조
└── state_machine.hpp # 상태 머신 (캐싱 정책)
```

10.3 주요 개념

- **Merkle Patricia Trie**: 이더리움 상태 트리 구조
- **Nibbles**: 4비트 단위 키 인코딩
- **RLP**: Recursive Length Prefix 인코딩
- **Keccak256**: 해시 함수
- **Compaction**: 디스크 공간 회수
- **Version Management**: 블록별 상태 히스토리

결론

TrieDB는 Merkle Patricia Trie 구조를 기반으로 다음과 같은 최적화를 통해 초고속 상태 관리를 구현합니다:

1. **효율적인 데이터 구조**: Nibbles 인코딩, 비트 마스크, 가변 크기 노드
2. **스마트 캐싱**: LRU 기반 가중치 캐싱, 레벨별 정책
3. **버전 관리**: 링 버퍼 기반 히스토리, 점진적 만료
4. **디스크 최적화**: 청크 기반 레이아웃, 비동기 I/O, Direct I/O

5. **압축 메커니즘**: 이중 리스트, 참조 카운팅, 자동 회수

이러한 최적화들이 결합되어 Monad는 초당 수만 건의 상태 업데이트를 처리하면서도 안정적인 성능을 유지합니다.